18/04

heapsort :

1. heapify the subtrees according to maxheap from last subtree to first subtree.
2. after heapification highest value is at root, swap with highes index decrement heap size .
3. disturnace created at first subtree, so again perform heapification according to max heap, so that next highest element comes to root swap with highest index
4. repeat above step(3) unitil we get heap with single data.

```
for(i=size/2-1;i>=0;i--)
{
        heapify(a,i,size);
}
for(i=size-1;i>0;i--)
{
        temp=a[0];
        a[0]=a[i];
        a[i]=temp;
        heapify(a,0,i);
}

void heapify(int *a,int pi,int size)
{
        int li,ri,large;
        large=pi;
        li=2*pi+1;
        ri=2*pi+2;
        if(li<size && a[li]>a[pi])
                large=li;
        if(ri<size && a[ri]>a[large])
                large=ri;
                                        if(large !=pi)
                                        {
                                                temp=a[pi];
                                                a[pi]=a[large];
                                                a[large]=temp;
                                                heapify(a,large,siz
                                        }
                                        }
```

TimeComplexity :
Bestcase : O(nlogn)
Averagecase : O(nlogn)
Worstcase : O(nlogn)
SpaceComplexity : O(1)

raj	70
sri	80
Jai	40
raj	60

Jai	40
raj	70
raj	60
sri	80

Jai	40
raj	60
raj	70
sri	80

	Best	Avg	Worst	Space	recursive / non recursive	In place	Stable	
Bubble	$O(n^2)$	$O(n^2)$	$O(n^2)$	1	non recursive	yes	✓	
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	1	non recursive	yes	X	
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	1	non recursive	yes	✓	
merge	$O(n\log n)$	$O(n\log n)$	$O(n\log n)$	$O(n)$	recursive	NO	✓	
Quick	$O(n\log n)$	$O(n^2)$	$O(n^2)$	$O(1)$	recursive	yes	X	
Heap	$O(n\log n)$			$O(1)$	recursive	yes	X	

cpp → 22 to 24 Sessions